SMART FILE & MEDIA MANAGER

Advanced Programming | Assessment 2 | Technical Report and Evaluation

Individual desktop application project — Python and PyQt6

Status of Report: Source Code, Automated Model/Scanner Tests and Metadata Benchmark (Linux) were developed and tested. GUI Execution of the PyQt6 GUI under Windows, Manual Evidence of GUI, Remote Git Activity and Live In-Class Demonstration are all left as items to be completed by the student on their own computer. The distinction between confirmed test results and tests yet to be done will help identify the difference.

Project	Summary
Theme	Local file and media processing manager
Platform	Windows 10/11 target; PyQt6 desktop GUI
Key techniques	QMainWindow; MVC-style separation; QThread; Qt signals; QtMultimedia; Python unittest; Git; PyInstaller build script
Assessment brief	Assessment 2: 15 tasks; 8 official deliverables; 70 available marks

1. Purpose, user and requirements

This tool was designed for individuals that want to organize and view metadata from multiple local files without having to open each file individually. Users can select a directory, perform a file scan, display metadata, apply filters to results, preview supported media types, and export a CSV inventory listing. Scans read the source files but do not modify nor upload those files.

ID	Functional requirement
FR01	Choose an existing local folder
FR02	Scan files with optional subfolders
FR03	Display sortable metadata table
FR04	Search and filter results
FR05	Preview common images and limited text
FR06	Preview audio with QtMultimedia
FR07	Show progress and cancel background processing
FR08	Export filtered data to CSV
FR09	Handle invalid paths, unreadable files and export failures

2. GUI plan and widget choices

The QMainWindow provides the following features (File/Help menu, Folder field and Browse action, Recursive scan checkbox, Scan and Cancel buttons, Search field, Category picker, Sortable Table Widget with preview area, Progress bar and Status bar). An additional feature of this application is that it utilizes a QSplitter allowing the user to dynamically adjust the amount of screen real estate used by the Results table versus the Preview window. As opposed to including widgets solely for visual interest, the application's controls support its primary workflow.

The first wireframe can be found here: docs/DESIGN.md. The table is the application's primary means of navigating through the data. When a user selects an item from the preview section, the application displays relevant information about the selected item. Any errors encountered are reported using a Message Dialog. In addition to displaying error messages via dialogs, the application also reports ordinary progress using the status bar.

3. Architecture and separation of concerns

Part	Responsibility / file
Model	app/models.py: immutable FileRecord data, categories, size/date formatting and filtering. app/scanner.py: folder discovery, metadata extraction and limited text preview.
View	app/view.py: QMainWindow, menus, layouts, table, image/text/audio preview, progress and user dialogs.
Controller	app/controller.py: connects GUI signals, starts/cancels workers, applies filters, exports CSV and coordinates updates.

Worker	app/workers.py: QObject moved to QThread; filesystem work performed away from GUI, results emitted via Qt signals.

Sequence of events: User clicks Scan → Controller verifies valid directory is provided → Worker initiated as QThread begins processing → Worker locates items, identifies type(s) of files processed and retrieves metadata → Signals report progress and results back to controller → Controller updates GUI view based on signals received from worker.

For all purposes related to manipulating GUI widgets the application has been designed so that interaction occurs within the context of the main thread.

4. Events, validation and exception handling

Browse, scan, cancel, table selection, search, category selection, audio preview and csv export are all tied together via qt signals.
The controller will verify if the target folder exists prior to scanning it.
The scanner will catch individual ose error messages while continuing to process other files. Errors are reported to the end-user as opposed to causing the entire scan to fail.
csv output will also catch write failure.

If there's a preview type that the system doesn't understand (preview), the system displays an explanation to the user.

5. Multimedia

QMediaPlayer with QAudioOutput from QtMultimedia is used for creating a QMediaPlayer object in order to create a short bundled wav notification to be played back by QSoundEffect once a scan is complete. The short bundled wav notification provides direct assistance to users who need to know when their long-running processes have completed. Availability of the codecs used to play the bundle wav notifications may vary depending upon the users operating system.

6. Concurrency and asynchronous processing

Executing the operations related to discovering folders and collecting metadata can take a significant amount of time on very large directories. Running these operations within a button callback would block the event-loop until they were completed. Therefore, the application runs the operation associated with finding new folders and gathering meta-data using a QThread called ScanWorker. The ScanWorker thread sends out progress, status, completed and finished signals. The progress-bar is set to "indeterminate" during the discovery phase of files. After the number of files is determined then the progress-bar is set to "determinate". In addition to displaying progress, the view allows the user to continue interacting with it. When the user clicks on the cancel button, the cancel button sends out a signal asking the worker-thread to safely stop its current activity.

7. Synchronisation and thread safety

Shared state in this program contains scan cancel status, and also what files appear in the GUI when scanning completes. An Event object from threading provides a safe way for the GUI to tell the worker to cancel the current operation. The worker never changes any of the GUI's widgets itself. Instead, after the list has been completed, it uses a signal; the controller owns the record objects being updated on the GUI thread. If either rule were violated, the operations may be reading an un-consistent state at some point or touching widget elements from a non-GUI thread, which would likely produce unpredictable behavior. Canceling a running scan will ask it to complete its run prior to closing the window.

8. Performance and parallelism investigation

Sequential vs. Threaded (Python) Metadata Inspect Benchmark
A simple repeatable benchmark compares sequential metadata access across Python 2.7 ThreadPoolExecutor.map with four threads versus sequential metadata access, both against a collection of 800 artificially created small local files and with three separate iterations. The amount of metadata accessed and total bytes accessed by both methods are identical. Time measurements were done with time.perf_counter() . While writing this package on my Linux system I have collected real-time performance data; however no claims about measurement accuracy are made regarding performance on your Windows machine.

Run	Sequential (s)	Threaded (s)
1	0.005571	0.033172
2	0.007782	0.082996
3	0.006739	0.029109
Average	0.006697	0.048426

The measured speedup provided by sequential vs. threaded was approximately 1/7 (0.14). Although this local test showed that using a threadpool was slightly slower, each stat call takes very little time and there is considerable overhead due to thread scheduling, submitting tasks and collecting results. This is an example of how parallelism does not always provide a faster solution

I/O-bound versus CPU-bound work

Most of reading file attributes is an IO / system call type of operation. Therefore, if you have many operations that are waiting (such as when accessing a very slow drive) then you might get some advantage by using

threads. However, with a small amount of data stored locally in your cache, then the overhead of thread creation/scheduling will likely dominate. The way that CPU-heavy loops work in Python is different: due to the Global Interpreter Lock (GIL), many Python threads will probably not run at the same time as other Python threads in CPU-bound processes, therefore for such tasks multiprocessing or writing your own native code would typically give better performance.

How scalable your solution will be will depend on how big your job/workload is, how fast your disks are, how many worker threads you use and the degree of contention there is among them. A parallel query can also be described as an executor.map where one operation is applied to all elements in the input list, but since you want to process the results together, it will need to be mapped back into a single object. A .NET Parallel LINQ dependency isn't necessary for this task.

Amdahl's Law

S(N) = 1 / ((1 − P) + P / N). If the parallelisable fraction P = 0.85 and four processors are available, S(4) = 1

/ (0.15 + 0.85/4) = **2.76×**. The serial 15% sets the limit even under ideal conditions. These P and N values are a hypothetical illustration, not parameters measured in the benchmark.

How to repeat the investigation

To perform your benchmarking, execute the following command:

python benchmarks/benchmark.py –-demo –-repeats 3 –-workers 4 — output evidence/own_benchmark.csv

For comparison purposes, repeat your testing using a larger version of the local directory used to generate these results and provide descriptions about what differs instead of just providing the output of running package-build-environment commands as if they were your personal results.

9. Test plan

The documentation included in docs/TEST_PLAN.md details a test plan which describes how each requirement will be tested under normal operating conditions, as well as under two abnormal conditions: empty directories and invalid/excessive input values. In addition, it allows space to enter the actual outcome and take screenshots for both the normal and abnormal condition test runs. Also, it makes a distinction between untested (manual) graphical user interface tests and the automatic tests.

10. Manual testing

Manual tests that are still pending in this package include: navigating through the UI, selecting files, viewing previews, opening empty directories, entering invalid path names for directories, attempting to open inaccessible files, playing back audio files, exporting failed scan attempts, performing a large scan (as opposed to a scan against a small set of files), canceling/scanning again while the application is scanning and closing the application before scanning has completed.

11. Automated testing

Thirty successful test runs of the Python unit tests (i.e., 30 passed; 0 failed), were made for the Qt independent code in the data/scanning/benchmark module. The test cases cover the categories of extensions, format boundaries, filters (case insensitive), safe formulas in CSV cells, recursive file discovery, hidden files, canceling tests, missing files, limits on text previews, and agreements between sequential and concurrent test runs. A copy of the original test run output can be seen in evidence/automated_tests.txt.

12. Debugging and corrective development

In addition to the test plan that was supplied there is also a defect/correction table which includes columns for problem, cause, solution, Git commit number, and rerun test results. There are no bugs from the GUI which have been fabricated to appear like they had occurred historically. When the student discovers some bugs while running the test suite on his/her Windows workstation he/she will need

to document those bugs and correct them. He/she will then need to provide documentation showing how he/she ran each of the tests again after the corrections had been implemented.

13. Git version control

The student will receive additional information about a local Git history to demonstrate the evolution of this students' development on a given project using logical commits and branch merges. The history must not be represented by the student as their own long term development. In addition to receiving an example of a local Git history of a student's project development, the student should create and document their own genuine commits based upon their development process, and illustrate meaningful branches and merge operations. Additionally, the student should demonstrate how they would resolve conflicts that may occur during the course of their development process (either through a controlled exercise or actual experience); remote push/pull/review should be demonstrated whenever possible and permitted within the institution; see docs/GIT_WORKFLOW.md

14. Distribution

This Package Provides A Dependency List And Run_Windows.bat To Create a Virtual Environment Using Python With Qt For Creating the GUI and Starting Main.py.
build_windows.bat Installs pyinstaller to create a Windows OneDir Distribution and Includes the Sound Asset.
This Is a Build Configuration Not Pre-Tested Executable.
You Must Verify That the Packaging/Execution Works On Your Windows Machine By Taking Screenshots. See docs/build_and_verification.md for More Information.

15. Evaluation and presentation

The Following Are The Strengths Of The Project (clear separation of projects, fast/responsive async scanner, cancel-safe scanner, useful metadata filter options, both media preview and notifications, csv export, reproducible tests).

The Following Are Some Limitations (memory usage when using large table widget, os dependent audio codec support, basic image preview no thumbnails), Lack of persistent indexing and that there is a separate cli for performance metrics rather than having it integrated into the application as a built in analysis view.

Some Improvements That Can Be Made Include Paged Models, Thumbnail Caching, An SQLite Index for Persistent Indexing, and A Configurable Scan Error Log.

Presentation Slides Are Located In: presentation

Final submission checklist

Deliverable	Location / status
Python/PyQt source	main.py + app/ — assembled; GUI must be run on Windows
Git history	git + docs/GIT_WORKFLOW.md — example local history; student activity required
Automated tests	tests/ + evidence/automated_tests.txt — 30 passed
Test plan/results	docs/TEST_PLAN.md — manual actual results pending
Technical documentation	docs/Technical_Report.pdf and docs/DESIGN.md
Performance study	benchmarks/benchmark.py + evidence/benchmark_results.csv/.txt — Linux demo measured
Build/distribution	build_windows.bat + requirements-build.txt — Windows packaging pending
README	README.md — setup and run instructions
Additional presentation	presentation/Project_Presentation.pptx — live demonstration pending

---

References (original documentation)

- Qt Documentation. QMainWindow. https://doc.qt.io/qt-6/qmainwindow.html
- Qt Documentation. QThread. https://doc.qt.io/qt-6/qthread.html
- Qt Documentation. QMediaPlayer. https://doc.qt.io/qt-6/qmediaplayer.html
- Qt Documentation. QSoundEffect. https://doc.qt.io/qt-6/qsoundeffect.html

- Python Software Foundation. concurrent.futures. https://docs.python.org/3/library/concurrent.futures.html

- Python Software Foundation. unittest. https://docs.python.org/3/library/unittest.html

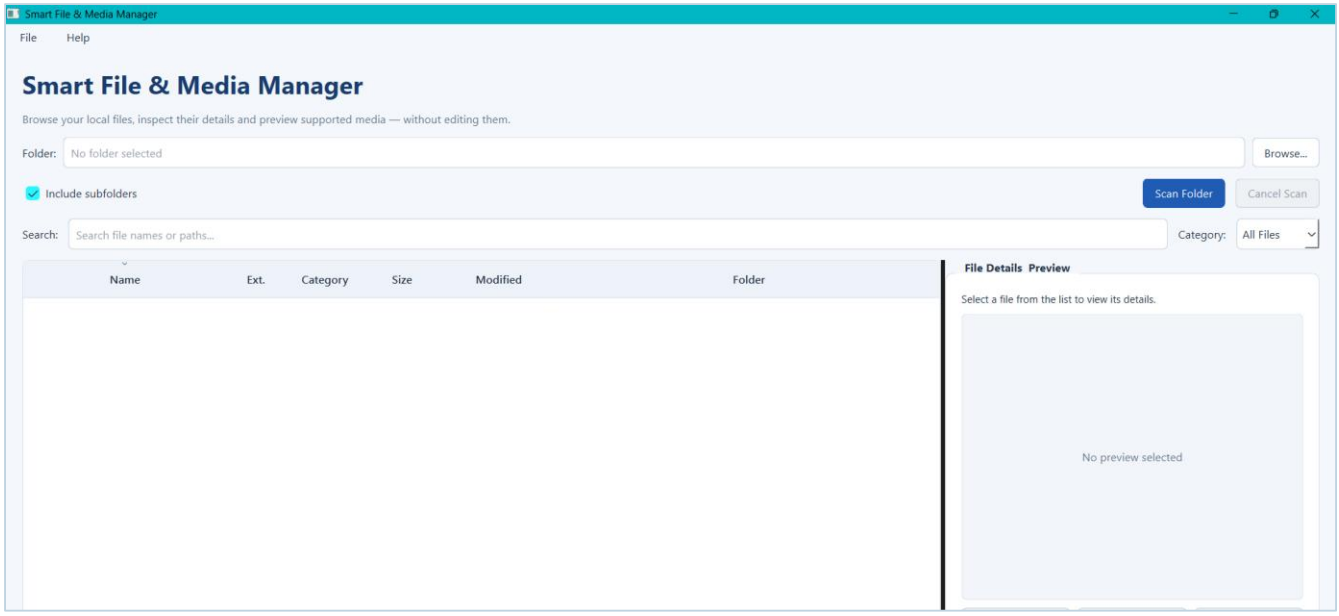
- Python Software Foundation. time.perf_counter. https://docs.python.org/3/library/time.html#time.perf_counter

- PyInstaller Documentation. https://pyinstaller.org/en/stable/

APPENDIX A | WINDOWS GUI EVIDENCE

Smart File & Media Manager - 1 of 4 screenshots

Figure A1. Initial application window



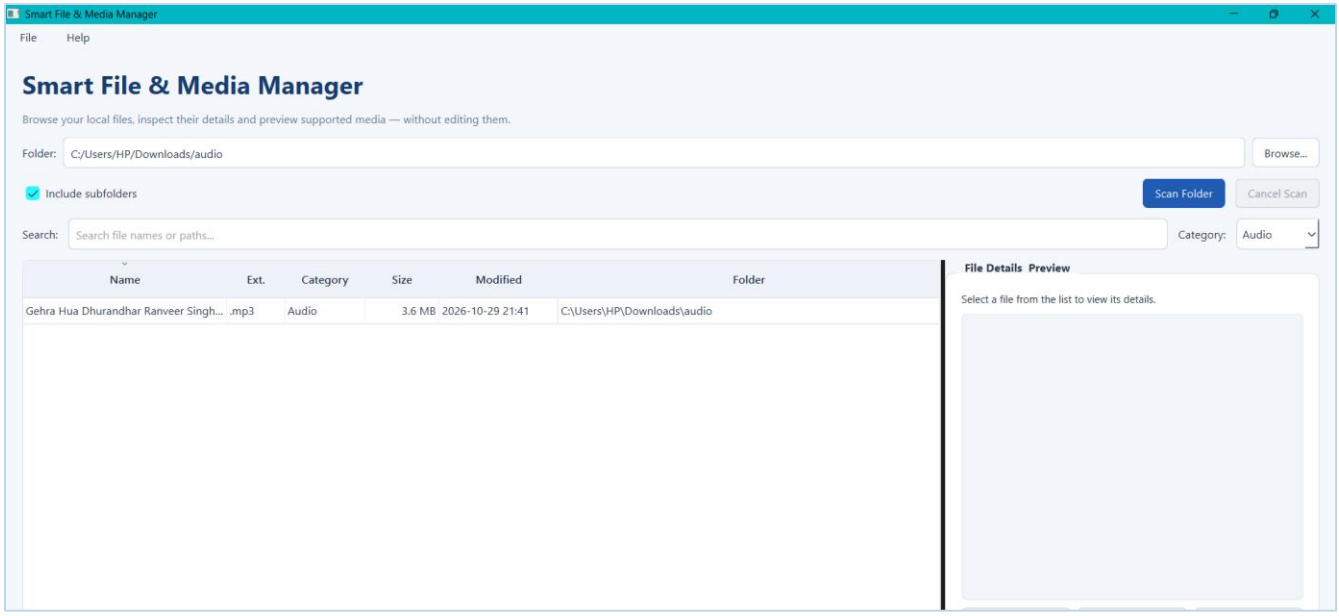
Screenshot observation

The QMainWindow opens with File and Help menus, the folder field, scan controls, category selector, results table, preview panel, status bar and progress indicator.

Evidence assessment

Supported: application GUI launches; layout and controls are visible.

Figure A2. Folder scan and audio filter



Screenshot observation

The selected Downloads/audio directory is shown. The Audio category displays one MP3 record with its name, extension, size and modification time.

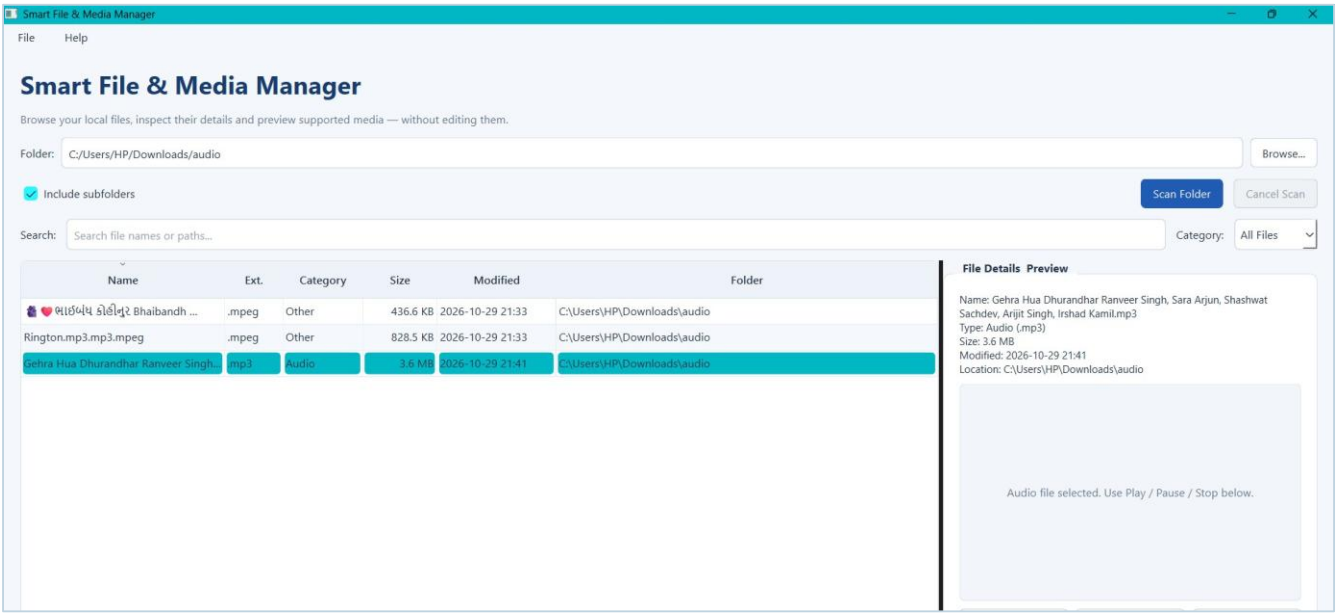
Evidence assessment

Supported: folder path, populated file list and category-filtered results are visible.

APPENDIX A | WINDOWS GUI EVIDENCE

Smart File & Media Manager - 3 of 4 screenshots

Figure A3. Selected MP3 and metadata



Screenshot observation

The all Files view in shows three are file records. Selecting the MP3 fills are the details panel with file name, type, size, timestamp and location; Play, Pause and Stop controls are available.

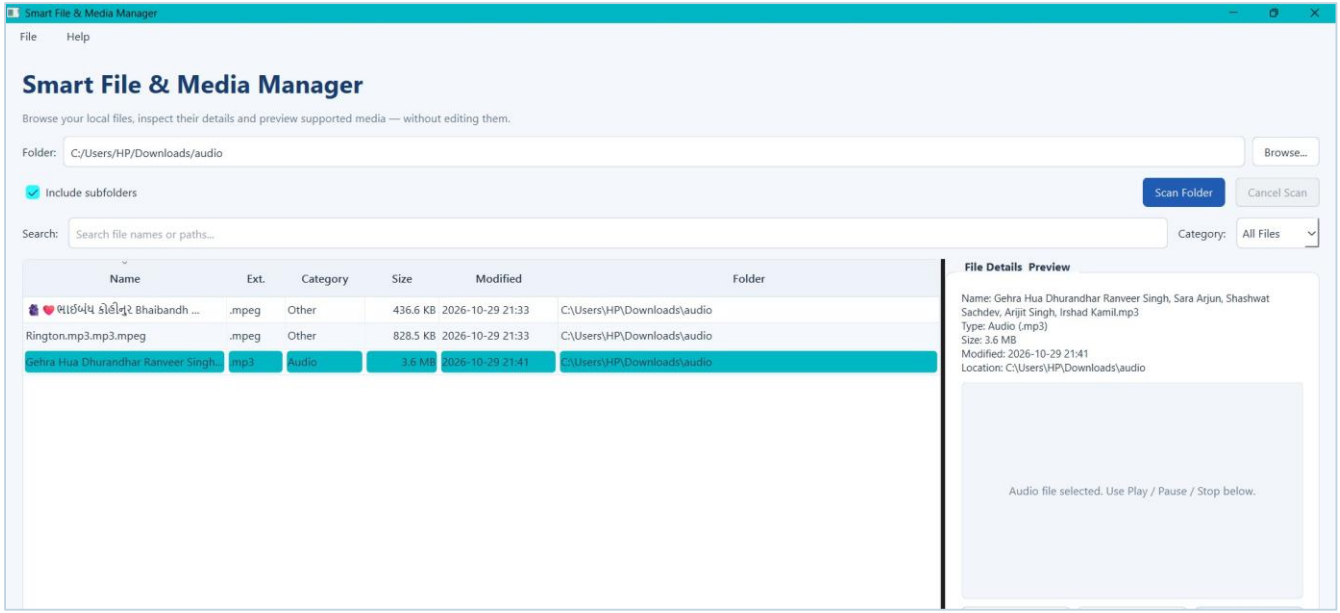
Evidence assessment

Supported: file selection, metadata display and audio preview interface. Not a proof of audible playback.

APPENDIX A | WINDOWS GUI EVIDENCE

Smart File & Media Manager - 4 of 4 screenshots

Figure A4. Audio preview control state



Screenshot observation

The selected audio file and the preview control remain the visible, with the Stop control shown in one different state.

Evidence assessment

Supported: controls and selection is visible. A screenshot cannot validate audio output, action timing or successful stop.